

M09 / LEARNER BOOK

ETL/ELT, dbt transformations and data quality

Standalone Zero-to-Job-Ready Data Engineer 2026

13 controlled lessons

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

START HERE How to use this module	3
DE09-01 ETL vs ELT and transformation boundaries	4
DE09-02 Raw, staging, clean/intermediate and serving contracts	7
DE09-03 Deterministic transformations and production SQL style	10
DE09-04 Deduplication and key integrity	13
DE09-05 Schema checks and data contracts	16
DE09-06 Source-to-target reconciliation	19
DE09-07 Incremental processing and safe reruns	22
DE09-08 Failure behavior, retries, quarantine and atomic state	25
DE09-09 Lineage, run metadata and auditability	28
DE09-10 Build and defend a reliable batch pipeline	31
DE09-11 dbt-style project anatomy, sources and dependency graph	34
DE09-12 Executable data tests, quality severity and CI boundaries	38
DE09-13 Incremental models, materialization, documentation and handoff	41

START HERE

How to use this module

Open the matching practice folder from the course root: 07_M09/practice. Work on a copy. Commands printed in a lesson use that module as the working folder unless the lesson names a different location.

Study in lesson order. After the last lesson, use Quick Revision, take the Module Revision Test, then open its separate Protected Answers. Gates are additional assessments at the boundaries shown in the course guide.

Build deterministic transformations with explicit contracts, key integrity, reconciliation, safe incremental behavior, operational audit evidence and a real local dbt project.

M08 taught you how to land source evidence safely. M09 starts from that handoff and asks a harder question: how do you transform data without losing the ability to prove that the result is correct?

The module uses one small retail-order pipeline repeatedly. The source files contain order versions. Your Python reference pipeline preserves raw evidence, validates the contract, normalizes fields, quarantines bad rows, handles duplicates, calculates net_sales, writes a serving table, records run metadata and advances state only after successful completion. A second local dbt/DuckDB project turns the same ideas into an explicit transformation graph with source(), ref(), tests, materializations and documentation.

This is deliberately not a tutorial-copy project. Every important claim has a check. In the clean first batch, six source rows become six serving orders worth 22,930.00. After the incremental batch, the serving table has nine orders worth 84,870.00. Replaying the same incremental batch applies zero new rows and the total stays 84,870.00. The duplicate fixture proves that one source batch can contain a safe latest version, an older discarded version and an ambiguous same-timestamp conflict that should be quarantined rather than guessed.

The Quick Revision Book and Module Revision Test are for later recall. They do not replace the learner book or practice. G04 is not after M09; it remains after M10 Docker + Airflow in the frozen course architecture.

DE09-01

ETL vs ELT and transformation boundaries

Watch first

[1. ETL vs ELT and transformation boundaries - retained visual](#)

OPTIONAL visual support - no independently verified bounded range. Learner book + official reference + supplied project/lab are the required route.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: Complete the prior M09 lesson (or M08 for DE09-01). Keep the matching review closed until you have saved an independent attempt.

[Instructor demonstration: DataTalksClub DE Zoomcamp - Introduction to Analytics Engineering](#)
[Optional](#) visual support: ETL/ELT boundary and analytics-engineering context. No independently verified bounded chapter range is assigned for this recording; do not watch the whole video just to complete this lesson. The learner book + local lab are the required route.

[Official/current reference: dbt Developer Hub](#)

Use rule: Watch/read for the idea and interface, then return to this book and the supplied project. Close the source before the independent task. The course project and expected controls below are authoritative for this module's assessment.

Transformation begins with a boundary decision

ETL and ELT describe where transformation happens relative to loading, not two rival religions. In ETL, data is extracted, transformed in a processing layer, then loaded to the target. In ELT, raw or lightly prepared data is loaded first and transformation happens inside the analytical platform. A real system can contain both patterns.

Picture a parcel entering a warehouse. One company may inspect and repack it before it reaches the warehouse shelf. Another may store the sealed parcel first, then open and organize it inside a controlled warehouse area. The important engineering question is not the acronym; it is which system owns raw evidence, transformation compute, state, failures and replay.

Follow the course pipeline once

The Python practice route is intentionally ETL-like: `batch_001.csv` is copied to raw evidence, validated/transformed in Python, then accepted rows are loaded into `orders_serving`. The local dbt route is ELT-like: `setup_raw_duckdb.py` first loads order versions to `raw.raw_orders`; dbt models then transform inside DuckDB.

Both routes must preserve the same invariants:

- source evidence stays traceable;
- accepted output grain is explicit;

- rejected data is not silently lost;
- reruns do not corrupt serving state;
- validation can reconcile source to target.

A concrete boundary mistake

Suppose you overwrite the only raw CSV with cleaned values before validation. The serving table may look correct, but you can no longer prove what the source actually sent. Or suppose you load everything into a warehouse and immediately expose the raw table to dashboards. You preserved evidence, but you failed the serving contract. ETL/ELT does not remove the need for layers.

Independent proof

Draw two architectures: one ETL-like route and one ELT-like route for these same order files. For each, state where raw evidence lives, where invalid rows go, where state advances, and which layer is safe for business users. Then answer: "If the transformation code is wrong tomorrow, which layer lets me rebuild without asking the source to resend?"

Changed retry

A source API charges for every reread and only retains seven days. Redesign your boundary so replay is possible after day seven. Explain why landing raw responses before transformation matters even if your main compute engine is a warehouse.

Compare two complete designs instead of memorizing acronyms

Suppose a nightly source delivers 20 GB of compressed order files. In an ETL-like design, a Python/Spark job may validate, normalize and reduce that data before loading curated tables. In an ELT-like design, the files may land in object storage/warehouse raw tables first and SQL/dbt transforms run where the data already lives. The first may reduce warehouse storage/compute on unusable rows; the second can preserve more source detail and exploit warehouse scale. Neither conclusion is automatic - network cost, data sensitivity, engine capabilities, source retention, governance and team skills matter.

A useful design review asks four questions. Evidence: can I reproduce what arrived? Compute: where is transformation cheapest/reliable? Failure: if transformation fails, what state is safe to keep? Replay: can I rerun without asking the source to resend? If your answer does not address those, saying "we use ELT" tells very little.

Common beginner mistakes and recovery

A beginner often calls any Python script ETL and any SQL query ELT without identifying where the durable load boundary actually is. Another mistake is cleaning data before preserving raw evidence because the cleaned file "looks better." If confused, draw boxes for source, durable raw, transform engine and serving target, then draw the first durable write. That usually makes the boundary visible.

Teach it back: explain one reason a company might prefer ETL-like preprocessing and one reason it might prefer ELT-like in-platform transformation. End with the sentence: "The acronym does not replace contracts, reconciliation or replay design."

Follow with me: prove both patterns on the same business data

First run the Python reference pipeline in a clean folder:

```
python -m practice.src.pipeline_reference practice/source/batch_001.csv \
  --workdir practice/output/m09_etl_demo --reset
```

You should see status: SUCCESS, six serving rows and serving_net_sales: 22930.0. In this route, validation/normalization/net-sales logic occurs before the accepted records are committed to the serving database. That is the ETL-like side of the comparison.

Now inspect the dbt project route. setup_raw_duckdb.py loads the raw order versions into raw.raw_orders first. Only after that do stg_orders, int_orders_deduped and fct_orders transform the data. That is ELT-like because the durable analytical database receives raw rows before model transformation.

The expected business result is not "ETL=22,930 and ELT=something else." If both implement the same contract over the same accepted business records, their comparable serving outputs should reconcile. The exercise is to move the transformation boundary while keeping evidence and correctness controls.

Before continuing, write one sentence for each route: what is durable before transformation? If you cannot answer that, you have not actually located the ETL/ELT boundary.

Common mistakes / recovery

- Calling ELT "load bad data and clean later." Raw landing still needs access/security/retention rules and
- downstream transforms still need tests.
- Calling ETL "safer" simply because transformation happens earlier. If the pre-load transform destroys
- source evidence, recovery can be worse.
- Choosing architecture from fashion instead of source retention, compute, governance and replay needs.

If stuck, draw the first durable copy and mark every place data can be lost. Then decide which system owns the transform and why.

Ready-made revision

ETL = transform before target load. ELT = load raw/near-raw first, transform in target platform. Neither automatically guarantees quality. The invariant is preserve evidence -> transform deterministically -> validate -> publish intentionally.

Evidence / mastery

Save the actual command/query/code, observed counts/values or error, one validation control, one short teach-back, and the changed retry. PASS -> continue to the next lesson. PARTIAL/UNCERTAIN -> targeted repair + changed retry.

DE09-02

Raw, staging, clean/intermediate and serving contracts

Watch first

[1. Raw, staging, clean/intermediate and serving contracts - retained visual](#)

OPTIONAL visual support - no independently verified bounded range. Learner book + official reference + supplied project/lab are the required route.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: Complete the prior M09 lesson (or M08 for DE09-01). Keep the matching review closed until you have saved an independent attempt.

[Instructor demonstration: DataTalksClub DE Zoomcamp - dbt Project Structure Video use: OPTIONAL visual](#) support. A reliable bounded chapter range was not independently verified for this source, so no fake timestamp is assigned. Use it only to visualize layers/project structure; the learner book + official reference are the required route.

[Official/current reference: dbt project configuration](#)

Use rule: Watch/read for the idea and interface, then return to this book and the supplied project. Close the source before the independent task. The course project and expected controls below are authoritative for this module's assessment.

Layers are contracts, not decorative folders

A reliable pipeline becomes easier to debug when each layer has one job. In this module we use four logical layers:

- raw: source evidence as received, with hashes/run metadata;
- staging: typed/normalized records that satisfy the immediate source contract;
- intermediate/clean: reusable business logic such as deduplication;
- serving: stable, documented outputs meant for downstream use.

Picture an airport. Raw is the arrivals hall: everyone who arrived is recorded. Staging is document/security inspection. Intermediate is baggage routing and connections. Serving is the departure gate for a specific flight. If you let an unverified arrival walk straight onto the plane, you skipped a contract boundary.

Use the actual order fields

The source header is fixed as: `order_id, updated_at, customer_id, region, product, qty,`

`unit_price, discount_pct, status`. Staging normalizes region/product text, parses UTC timestamps, converts quantity and money fields, rejects impossible values and calculates no serving claim yet. The serving table adds deterministic `net_sales` and a `source_run_id` so each row remains traceable.

Raw and serving have different promises. Raw may contain malformed rows because preserving evidence is its purpose. Serving should not contain a row whose key, schema or monetary rule is unresolved. "Raw has bad data" is not automatically a failure; "serving silently contains bad data" is.

What not to do

Do not use DISTINCT as a universal cleaning layer. It can hide duplication without explaining which record won. Do not let staging overwrite raw. Do not make every temporary calculation a permanent table. A layer exists because it owns a meaningful contract or reuse boundary.

Follow with one source row

O6003 arrives with region text east . Raw keeps that exact evidence. Staging normalizes the region to East. Serving calculates $1 * 12000 * (1 - 0.05) = 11400.00$. If somebody questions the normalized region later, you can compare all three states instead of guessing what happened.

Independent proof

For O6003, write a four-row lineage note: raw value, staging value, intermediate decision and serving value. Then inspect the actual runtime files after running batch 001. State which files could be deleted/rebuilt and

which are evidence you should preserve.

Changed retry

A new source field sales_channel appears. Decide whether raw, staging and serving should all change immediately. Raw can preserve it immediately; staging/serving should change only after the contract and downstream meaning are deliberately updated. Explain that difference in your own words.

Layer contracts should stop accidental coupling

Imagine a dashboard query reads directly from a staging table because "the columns are already there." Next week staging changes a field name to match the upstream contract and the dashboard breaks. The problem was not the rename alone; the consumer depended on a layer that never promised long-term stability. Serving models exist so downstream users depend on an intentional contract rather than whatever intermediate shape happens to exist.

Conversely, do not turn every two-line calculation into a permanent intermediate table. Each persisted layer adds storage, ownership and rebuild obligations. Create a layer when it protects a contract, isolates a grain change, enables reuse, improves observability or makes recovery clearer.

Recovery ladder

If you cannot decide where a rule belongs, ask: Is this preserving source evidence? -> raw. Is this parsing/standardizing source semantics? -> staging. Is this reusable transformation or grain logic? -> intermediate. Is this a consumer-facing stable definition? -> serving. If two answers seem possible, document the trade-off and choose one owner instead of duplicating logic.

Teach it back: why is raw allowed to be ugly? Why should staging normally be rebuildable? Why is serving not necessarily "the final file" but a contract for consumers?

Follow with me: inspect the layer handoff in one successful run

After the first-batch run, open these three things: the raw run folder and `_manifest.json`, the staging JSONL for the same run ID, and `orders_serving` in `serving.sqlite`.

Raw should still show the source-shaped evidence. Staging should show normalized records such as `O6003 region=East` and `O6004 product=Keyboard`. Serving should contain the business-facing `net_sales` plus

```
source_run_id.
```

Use the inspection helper:

```
python practice/src/inspect_serving.py practice/output/m09_etl_demo/serving.sqlite
```

Confirm `O6003` is 11,400.00 and `O6005` is 6,300.00. The raw file should still contain `east` and `Keyboard` with their original spaces. That contrast is the point: normalization changed the interpreted representation while raw evidence remained unchanged.

Now imagine the staging file disappeared. Because raw evidence and code still exist, staging is rebuildable. Imagine the raw copy disappeared after the source retention expired. You may still have serving values, but you have lost the strongest evidence for replay/audit. This is why "which layer can be rebuilt?" is a useful design question.

Common mistakes / recovery

- Reading serving directly from raw because a sample query happens to work.
- Treating quarantine as a fifth business layer; it is evidence about rejected records, not a curated output.
- Copying the same cleansing rule into staging and serving, creating two owners.

If a downstream number is wrong, walk backward serving -> intermediate -> staging -> raw and compare the first layer where key/value/count differs.

Ready-made revision

A layer is useful when it has a different promise. Raw preserves truth about arrival. Staging enforces source shape/types. Intermediate expresses reusable logic. Serving exposes a stable downstream contract.

Evidence / mastery

Save the actual command/query/code, observed counts/values or error, one validation control, one short teach-back, and the changed retry. **PASS** -> continue to the next lesson. **PARTIAL/UNCERTAIN** -> targeted repair + changed retry.

DE09-03

Deterministic transformations and production SQL style

Watch first

[1. Deterministic transformations and production SQL style - retained visual](#)

OPTIONAL visual support - no independently verified bounded range. Learner book + official reference + supplied project/lab are the required route.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: Complete the prior M09 lesson (or M08 for DE09-01). Keep the matching review closed until you have saved an independent attempt.

[Instructor demonstration: DataTalksClub DE Zoomcamp - dbt Models Video](#) use: [OPTIONAL visual support](#). No independently verified bounded chapter range is assigned. Use it only for SQL-model/dependency visuals; the learner book + official reference + local lab are the required route.

[Official/current reference: dbt SQL models](#)

Use rule: Watch/read for the idea and interface, then return to this book and the supplied project. Close the source before the independent task. The course project and expected controls below are authoritative for this module's assessment.

Determinism means the same business inputs produce the same business result

Production transformation code should not change its answer because row order happened to differ, the machine clock changed, floating-point rounding drifted, or a query relied on an unspecified tie. Determinism is what makes reruns, debugging and review believable.

The course pipeline uses explicit field names, explicit UTC parsing, Decimal for monetary calculation, a fixed rounding rule and stable keys. `net_sales` is calculated as `qty * unit_price * (1 - discount_pct)` and rounded to two decimal places. For O6001, the result is $2 * 850 * 0.90 = 1530.00$. For O6005, it is $2 * 3500 * 0.90 = 6300.00$.

$3500 * 0.90 = 6300.00$.

Production SQL style is part of correctness

Readable SQL is not cosmetic when the transformation has multiple grains. Prefer explicit columns, named CTEs, visible filters and deterministic window ordering. A reviewer should be able to say what one row represents after each step. Avoid `SELECT *` in stable models because upstream column drift can silently change downstream shape.

A common anti-pattern is:

```
select *  
from raw_orders
```

```
qualify row_number() over (partition by order_id order by updated_at desc) = 1
```

It looks concise, but if two conflicting versions have the same `updated_at`, the winner is not justified. A deterministic query needs either a trustworthy secondary ordering key or a rule that rejects the ambiguity.

Follow the first batch

After normalization, the six accepted order IDs remain O6001-O6006. The serving total is 22,930.00. Reorder the source CSV rows and rerun in a clean work directory: the serving keys and total should remain the same. Output-file line order may be chosen for readability, but business membership and values must not depend on arrival order unless the contract says they should.

Break it on purpose

Change a money calculation to binary float and remove explicit rounding. On tiny fixtures you may still see familiar numbers; that does not prove the approach is safe. Next remove deterministic sorting before a human-readable export. The database result may still be mathematically correct, but diffs become noisy and review harder. Determinism includes both business rules and reproducible evidence.

Independent proof

Write a tiny function/query that calculates `net_sales` for the first batch. State the input grain and output grain. Shuffle the input order and prove key/value parity. Then explain why deterministic business results matter more than identical physical file bytes.

Changed retry

You receive two records for one order at the same timestamp. You are not allowed to invent a winner from file order. Write the extra source-owned field or quarantine rule you would need before calling the transformation deterministic.

Determinism also applies to joins and windows

Suppose a model joins orders to a customer-history table that has two rows for one customer. If you expected one customer row, the join can multiply an order. The SQL is syntactically valid and repeatable, yet the business transformation is wrong because the source grain assumption was false. Deterministic code still needs correct cardinality.

Window functions introduce another trap. `row_number() over(partition by order_id order by updated_at desc)` is deterministic only if `updated_at` establishes a unique winner for each order. If ties exist, add a source-owned sequence or reject the ambiguity. Adding `order by random()` or relying on physical insertion order does not make the business rule legitimate.

Validate from two directions

For money, recalculate a few rows by hand and compare aggregate totals. For keys, count rows and distinct business keys. For reruns, compare final business values after shuffling input order. For SQL, explicitly state input and output grain before trusting the query. These controls catch different failure modes.

Stuck? Reduce the model to three rows that reproduce the ambiguity. Write the expected winner rule in plain English before editing code. If you cannot state a business rule, do not hide the

uncertainty in syntax.

Follow with me: calculate and then challenge one transformation

Take O6001 and O6003 before running anything. Predict:

```
O6001 = 2 * 850 * (1 - 0.10) = 1530.00  
O6003 = 1 * 12000 * (1 - 0.05) = 11400.00
```

The reference pipeline uses Decimal and explicit two-decimal rounding. After the run, compare these predictions with serving values.

Next make a disposable copy of batch_001.csv and reverse the data-row order. Run it in a fresh work directory. The target business keys/values and total should still be six rows / 22,930.00. If reversing input changes the business result, some transformation depends on physical order.

For SQL, practise the same discipline. Before writing a window function, write the intended grain above it as a comment. Before a join, record expected key/cardinality. After the query, compare row count and distinct business-key count. "Query returned rows" is not a validation.

A professional reviewer should be able to read the transformation and answer: which source fields are used, which types/rounding are intentional, how ties are resolved, and what independent check proves the result. If those answers exist only in your head, the transformation is harder to maintain.

Common mistakes / recovery

- SELECT * in stable models, implicit casts, unordered tie resolution and using current time inside historical
- transforms.
- Assuming repeatable syntax implies correct grain/cardinality.
- Rounding only in the final report while intermediate money math drifts.

Recovery: reduce to a tiny fixture, state input/output grain, predict rows/values, run, then compare with an independent count/key/money control.

Ready-made revision

Deterministic transformation = explicit inputs + explicit types + explicit grain + stable keys/order where needed + controlled rounding + no hidden wall-clock/file-order dependence.

Evidence / mastery

Save the actual command/query/code, observed counts/values or error, one validation control, one short teach-back, and the changed retry. PASS -> continue to the next lesson.

PARTIAL/UNCERTAIN -> targeted repair + changed retry.

DE09-04

Deduplication and key integrity

Watch first

[1. Deduplication and key integrity - retained visual](#)

OPTIONAL visual support - no independently verified bounded range. Learner book + official reference + supplied project/lab are the required route.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: Complete the prior M09 lesson (or M08 for DE09-01). Keep the matching review closed until you have saved an independent attempt.

[Instructor demonstration: DataTalksClub DE Zoomcamp - dbt Tests Optional visual support:](#)

uniqueness/not-null testing context. No independently verified bounded chapter range is assigned for this recording; use the learner book and local tied-duplicate lab for the actual deduplication contract.

[Official/current reference: dbt data tests](#)

Use rule: Watch/read for the idea and interface, then return to this book and the supplied project. Close the source before the independent task. The course project and expected controls below are authoritative for this module's assessment.

Deduplication is a business decision about identity and version precedence

Duplicate-looking rows can mean several different things: an exact resend, an older version, two legitimate line items, or a true conflict. The first question is therefore what key identifies one business record at this stage? In this project the serving grain is one row per `order_id`.

The duplicate fixture is designed to stop you from using `DISTINCT` blindly. It contains four source rows:

- O6101 at 08:00 with qty 1;
- O6101 at 09:00 with qty 2;
- O6102 at 08:30 with qty 1;
- O6102 at 08:30 with qty 2.

For O6101, the later timestamp gives a defensible latest version: stage the 09:00 row and record the older row as discarded history. For O6102, both candidate "latest" rows have the same timestamp but different payloads. File position is not a trustworthy business rule, so both conflicting maxima are quarantined.

Expected evidence

The fixture reconciles as: 4 source = 1 staged + 2 quarantined + 1 older discarded. The staged row is O6101 qty 2, worth 1700.00. The quarantine reason for O6102 is

ambiguous_latest_duplicate.

That result is much more informative than `SELECT DISTINCT`, which would keep both O6102 rows because they are not identical, or a loose `row_number()` that would choose one without explaining why.

Key integrity after deduplication

Once a row is accepted into serving, `order_id` must be unique. That is not merely a database preference; it is evidence that the stated grain is being enforced. A uniqueness test cannot tell you whether the chosen row was semantically correct, so dedup policy and key test are both required.

Independent proof

Run the duplicate fixture in a clean work directory. Save the source count, staged count, quarantine count, discarded count and serving row. Explain why O6101 is safe to resolve and O6102 is not. Then write the SQL window pattern you would use only when a trustworthy secondary sort key exists.

Changed retry

Add `source_sequence` supplied by the source system and guaranteed unique per order change. Now both same-timestamp O6102 rows have different source sequences. State whether that field is strong enough to resolve the conflict and why a local CSV line number would be weaker.

Why `DISTINCT` and arbitrary row numbers are dangerous repairs

`DISTINCT` removes exact duplicate result rows; it does not decide which of two conflicting versions is correct. `ROW_NUMBER()` can choose one row, but the `ORDER BY` inside the window is the business policy. If that ordering is incomplete, the database may return a stable-looking result in one run and still have no defensible winner rule.

In the duplicate fixture, O6101 demonstrates version precedence while O6102 demonstrates identity conflict. Those are intentionally different. Treating both with the same "keep last row" rule would erase the signal that your source contract is insufficient for O6102.

Production extension

Real pipelines may carry a change-sequence, source commit LSN, event offset, ingestion timestamp or effective-from timestamp. Each can be a valid tie-breaker only if its semantics are trustworthy for the business question. A local ingestion timestamp can tell which record arrived later, but that does not always tell which source change is newer.

Stuck? Group the conflicting rows by business key. Sort only by fields whose precedence you can explain. If more than one candidate remains at the maximum precedence and payloads differ, quarantine/alert rather than inventing a winner.

Follow with me: run the duplicate trap and account for every row

Use a fresh work directory and a higher temporary quarantine threshold so the fixture can complete while still preserving conflict evidence:

```
python -m practice.src.pipeline_reference practice/source/batch_duplicates.csv \
  --workdir practice/output/m09_dups --reset --threshold 0.60
```

Expected controls are four source rows, one staged row, two quarantined rows, one discarded older version, one serving row and 1,700.00 net sales.

Open the quarantine JSONL. Both O6102 candidates should be present with reason `ambiguous_latest_duplicate`. This matters because the pipeline did not simply throw away one candidate. The ambiguity is visible and reviewable.

Now inspect O6101. Qty1 at 08:00 is older than qty2 at 09:00, so the later row is a defensible winner under the fixture's "latest updated_at" contract. Explain why the same rule cannot choose between O6102 rows: their precedence fields tie while their payloads disagree.

Finally compare three possible "repairs": `DISTINCT`, `row_number()` with incomplete ordering, and quarantine. Only quarantine is honest when the contract cannot identify a winner. The right long-term repair is usually a stronger source-owned version/change sequence, not a more clever local guess.

Common mistakes / recovery

- Deduplicating before defining the business key.
- Keeping "the last row" when last means only physical file order.
- Counting quarantined conflicts as successful dedup.

Recovery: group by business key, list candidate versions, identify the maximum trusted precedence, then check whether more than one conflicting maximum remains. Ambiguity is an outcome you may need to preserve.

Ready-made revision

Dedup = define identity -> define version precedence -> detect ambiguity -> quarantine unresolved conflicts -> enforce target uniqueness -> reconcile every source row to accepted/quarantined/discarded evidence.

Evidence / mastery

Save the actual command/query/code, observed counts/values or error, one validation control, one short teach-back, and the changed retry. PASS -> continue to the next lesson.

PARTIAL/UNCERTAIN -> targeted repair + changed retry.

DE09-05

Schema checks and data contracts

Watch first

[1. Schema checks and data contracts - retained visual](#)

OPTIONAL visual support - no independently verified bounded range. Learner book + official reference + supplied project/lab are the required route.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: Complete the prior M09 lesson (or M08 for DE09-01). Keep the matching review closed until you have saved an independent attempt.

[Instructor demonstration: Confluent - How to use Data Contracts for Long-Term Schema](#)

[Management](#) Optional visual support: explicit schema/data-contract context. A reliable bounded chapter endpoint was not independently verified, so no timestamp is assigned here; the learner book + changed-drift lab are the required route.

[Official/current reference: dbt source properties](#)

Use rule: Watch/read for the idea and interface, then return to this book and the supplied project. Close the source before the independent task. The course project and expected controls below are authoritative for this module's assessment.

A data contract turns "we hope the file looks the same" into an explicit rule

The source schema in this module is not inferred anew on every run. The pipeline expects nine named columns in a defined order. That strictness is intentional for the exercise because a renamed/missing field can change business meaning.

batch_schema_drift.csv changes region to sales_region and omits discount_pct. A permissive script might fill the missing discount with zero and continue. That would silently increase some future order values. The reference pipeline instead preserves the raw file, records the observed header, marks the run FAILED_CONTRACT, and does not advance state.

Compatibility is a decision, not a parser trick

Not every schema change must fail forever. An additive nullable column may be backward compatible for one consumer. A renamed key field may be breaking. A decimal-to-text change may be acceptable only with a proven conversion rule. The contract should identify owner, version, expected fields/types/semantics and what happens when the source changes.

Time semantics belong in the contract too. updated_at in this module must be UTC with Z. An offset-less timestamp such as 2026-09-04 09:00:00 is not silently assumed to be UTC. Guessing time zones changes incremental ordering and can skip or replay data.

Raw evidence comes before rejection

Notice the order of operations: copy/hash raw input first, then check the header. A failed contract still deserves an immutable receipt. If you reject before preserving evidence, debugging becomes a conversation about what you think the source sent.

Independent proof

Run the normal first batch, then the schema-drift batch in the same work directory. Prove that the normal state remains the last successful state. Save expected vs observed columns from the failed manifest. Write a one-paragraph migration proposal that would intentionally support sales_region without pretending the old contract never existed.

Changed retry

The source adds a nullable campaign_id column while keeping all nine original fields. Decide whether your strict equality check should be relaxed to a compatibility policy. State what staging/serving changes are required before exposing the new field downstream.

Contract evolution needs compatibility reasoning

A field can be technically parseable and still be semantically breaking. Changing discount_pct from 0.10 to 10 without changing the type from numeric is catastrophic even though every parser still succeeds. Contracts therefore need units/semantics, not only data types.

Think in three compatibility categories. Backward compatible: existing consumers can still process new data (for example, an optional additive field they ignore). Breaking: existing logic cannot interpret the new shape/meaning safely. Unknown: you do not yet have enough evidence, so quarantine/fail until the owner clarifies. "We can cast it" is not a compatibility proof.

Version and ownership

A contract should identify who can approve changes and how versions are introduced. For a breaking source change, you may temporarily support v1 and v2 side-by-side, map both into one staging contract, then retire v1 after evidence shows the migration is complete. Silent in-place reinterpretation makes historical replay ambiguous.

Teach it back: give one schema change that is additive, one that is breaking and one semantic change that would not be detected by column/type comparison alone.

Follow with me: prove that schema failure does not contaminate successful state

Start by loading the clean first batch into a fresh work directory. Record its processed-hash state and serving controls. Then run:

```
python -m practice.src.pipeline_reference practice/source/batch_schema_drift.csv \
  --workdir practice/output/m09_contract
```

Expected status is FAILED_CONTRACT. The observed columns contain sales_region and omit discount_pct; the expected columns still contain region and discount_pct.

Now inspect state.json. It should still describe only the prior successful batch. Inspect serving: it should still be six rows / 22,930.00. Inspect the drift run's raw manifest: the file was preserved even though transformation did not proceed.

This gives three independent facts: receipt succeeded, contract failed, business state did not advance. Many weak pipelines collapse these into one boolean "job failed" and lose the ability to reason about recovery.

For a real schema migration, do not edit the old run's manifest. Version the mapping/contract, test a changed fixture, and record when the new contract becomes active. Historical evidence should remain interpretable under the rule that existed at the time.

Common mistakes / recovery

- Adding defaults for missing fields without owner approval.
- Treating type compatibility as semantic compatibility.
- Advancing state because the raw file was successfully copied even though transformation contract failed.

Recovery: save raw + observed schema, compare against versioned expectation, classify additive/breaking/unknown, then change mapping only with an explicit rule and changed-data test.

Ready-made revision

A contract includes more than column names: identity, types, required/null rules, time semantics, allowed values, ownership/version and failure behavior. Unknown change -> preserve raw -> detect -> quarantine/fail deliberately -> update contract intentionally.

Evidence / mastery

Save the actual command/query/code, observed counts/values or error, one validation control, one short teach-back, and the changed retry. PASS -> continue to the next lesson.

PARTIAL/UNCERTAIN -> targeted

repair + changed retry.

DE09-06

Source-to-target reconciliation

Watch first

[1. Source-to-target reconciliation - retained visual](#)

OPTIONAL visual support - no independently verified bounded range. Learner book + official reference + supplied project/lab are the required route.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: Complete the prior M09 lesson (or M08 for DE09-01). Keep the matching review closed until you have saved an independent attempt.

[Instructor demonstration: DataTalksClub DE Zoomcamp - dbt Sources Optional visual support: dbt](#) sources/freshness context. No independently verified bounded chapter range is assigned; reconciliation proof remains course-controlled in the learner book and local project.

[Official/current reference: dbt sources](#)

Use rule: Watch/read for the idea and interface, then return to this book and the supplied project. Close the source before the independent task. The course project and expected controls below are authoritative for this module's assessment.

Reconciliation answers: where did every row and every important measure go?

A pipeline can finish successfully and still be wrong. Reconciliation compares independent controls across boundaries so silent loss, duplication or monetary drift becomes visible.

The first row-count equation in this module is: $\text{source_rows} = \text{staged_rows} + \text{quarantine_rows} + \text{dedup_discarded}$. For batch 001: $6 = 6 + 0 + 0$. For the duplicate fixture: $4 = 1 + 2 + 1$. These equations force every source row into an explained bucket.

Money is a second independent control

The clean first serving state contains six rows totaling 22,930.00. After batch 002, O6002 is updated and O6007-O6009 are added. The serving table contains nine rows totaling 84,870.00. The change is 61,940.00: 450 increase to O6002 plus 52,250 + 3,240 + 6,000 from the new orders.

Why keep count and money checks separate? A duplicated zero-value row can break count while leaving money unchanged. A missing expensive row can break both. Two wrong values can even offset and preserve a grand total. No single metric proves correctness.

Reconcile at the right grain

If raw is one row per order version and serving is one row per current order, raw row count does not have to equal serving row count after deduplication/incremental history. The equation must account for the grain change. "Counts differ" is not automatically a bug; unexplained counts are.

Independent proof

For batch 001 and batch 002, calculate source row counts before running the pipeline. After each run, inspect the manifest and serving database. Reproduce 22,930.00 and 84,870.00 independently from the source data. Then write why the serving row count changes from 6 to 9 although four incremental source rows arrived.

Changed retry

Imagine one incremental row updates an existing order downward by 5,000 while two new orders add 5,000 total. The serving grand total would not change. Design two controls that still detect the transformation activity.

Reconciliation must survive updates, not only inserts

Batch 002 contains four source rows but serving grows from six to nine, not ten, because O6002 is an update to an existing business key. If you expected target rows to increase by source rows every time, your control would falsely flag a correct update. Reconciliation must understand operation type and grain.

A stronger incremental control can decompose the effect: one existing key updated + three new keys inserted. Then validate the net monetary movement. O6002 changes from 1350 to 1800 (+450), while O6007/O6008/O6009 add 61,490, producing total movement 61,940. This gives you a story that matches both keys and money.

Tolerance needs a reason

Financial pipelines sometimes allow tiny rounding tolerances; row-count equality normally should not use a fuzzy tolerance. If you permit a difference, document the reason and maximum. "Close enough" without a contract converts an error into an accepted mystery.

Stuck? Reconcile in layers: source receipt -> accepted/rejected/discarded -> target insert/update/ignored -> final target control. The first layer where the equation stops balancing is where to investigate.

Follow with me: reconcile row movement and money movement separately

Run batch 001 and then batch 002 in the same clean runtime. Before batch 002, serving contains six keys. After it, there are nine because one incoming key (O6002) updates an existing row and three keys are new.

Make a small control table:

Before rows: 6	Before net: 22930.00
Incoming rows: 4	Existing updated keys: 1
New keys: 3	After rows: 9
After net: 84870.00	Net movement: 61940.00

Then reconstruct the movement:

O6002 old 1350 -> new 1800	+450
O6007	+52250
O6008	+3240
O6009	+6000
Total movement	+61940

The row-count equation and money equation tell different stories and agree on the business effect.

Now compare the duplicate fixture. There, target rows cannot equal source rows because dedup/quarantine intentionally changes grain/membership. Your reconciliation must include those buckets. Write controls that express the expected logic rather than forcing every pipeline into `source_count == target_count`.

Common mistakes / recovery

- Using one grand total as the only proof.
- Comparing counts across different grains without accounting for updates/dedup/filtering.
- Letting quarantine/discard buckets disappear from the equation.

Recovery: build a control table with source, accepted, rejected, discarded, insert/update/ignored and final target measures. Investigate the first equation that fails rather than patching the final number.

Ready-made revision

Reconcile counts + keys + important measures + state across each grain change. Every difference must be explained by a deliberate rule such as quarantine, deduplication, update or filter.

Evidence / mastery

Save the actual command/query/code, observed counts/values or error, one validation control, one short teach-back, and the changed retry. PASS -> continue to the next lesson.

PARTIAL/UNCERTAIN -> targeted repair + changed retry.

DE09-07

Incremental processing and safe reruns

Watch first

[1. Incremental processing and safe reruns - retained visual](#)

OPTIONAL visual support - no independently verified bounded range. Learner book + official reference + supplied project/lab are the required route.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: Complete the prior M09 lesson (or M08 for DE09-01). Keep the matching review closed until you have saved an independent attempt.

[Instructor demonstration: DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses with dlt](#)
[Optional](#) visual support: incremental-loading concepts. No independently verified bounded chapter range is assigned; the book and local rerun/state lab provide the required proof.

[Official/current reference: dbt incremental models](#)

Use rule: Watch/read for the idea and interface, then return to this book and the supplied project. Close the source before the independent task. The course project and expected controls below are authoritative for this module's assessment.

Incremental processing is a promise about what you will not miss on rerun

A full rebuild rereads/recomputes everything. An incremental pipeline processes only data that could change the target. The speed benefit is attractive, but the correctness burden is higher because the pipeline carries state across runs.

The Python project uses processed source hashes as one idempotency/state control. Batch 001 succeeds and its hash is recorded. Batch 002 succeeds and its hash is added. Replaying batch 002 with replay mode intentionally reprocesses its four staged records, but serving applies zero changes because the same/current versions are already present. Serving remains nine rows and 84,870.00.

File identity and record watermark solve different problems

A processed file hash prevents accidental double-processing of the exact same file in the normal route. A record watermark such as (updated_at, order_id) helps query a changing source incrementally. Neither is universal. A corrected file with the same name but different bytes should not be mistaken for the old file. A source that sends a late update older than your high-water mark needs CDC, a lookback or another source contract.

Safe rerun means idempotent effect, not "skip everything"

Sometimes the correct rerun is to skip a file already processed. Sometimes it is to rebuild a partition. Sometimes it is to reread a safe lookback and upsert by key. The invariant is that

repeating the run does not multiply business results or move state incorrectly.

dbt preview

The later dbt incremental model uses `unique_key='order_id'` and re-reads rows at the current maximum timestamp using `>=`. This protects a new row tied exactly at the max timestamp from being skipped. It still does not solve arbitrarily late historical updates. DE09-13 returns to this limitation.

Independent proof

Run batch 001, batch 002 and then replay batch 002. Save all three manifests and serving controls. Explain why the third run can stage four rows but apply zero business changes. Then state what evidence would convince you a true late update from 2026-09-01 was not silently missed.

Changed retry

A source corrects O6001 two days later but leaves `updated_at` unchanged. Explain why a pure timestamp watermark cannot discover the correction. Propose a stronger source contract such as CDC sequence/change version, file snapshot reconciliation or periodic lookback/full comparison.

State is data and deserves the same care as the business table

A checkpoint, watermark or processed-hash list is not disposable bookkeeping. If it is wrong, the next run can skip data or reprocess too much. Store state durably, write it atomically and make it inspectable. The module writes JSON through a temporary file then replaces the state file so a partially written checkpoint is less likely.

State should also describe what it means. A timestamp watermark without a tie-breaker is ambiguous. A file hash list means "these exact bytes completed successfully," not "all business records up to this date are complete." The operator needs to know the semantic boundary before editing or resetting state.

Safe backfill pattern

A historical backfill should normally use an explicit scope and avoid corrupting the normal forward checkpoint. Rebuild a target partition or upsert known keys, then reconcile. Do not move the live watermark backward and hope normal processing recovers correctly unless that behavior is designed/tested.

Teach it back: explain why rerunning is not automatically the same as replaying, and why a backfill should usually have its own explicit scope/evidence.

Follow with me: replay without multiplying the target

Run clean batch 001 and batch 002. Then run batch 002 again with `--replay`:

```
python -m practice.src.pipeline_reference practice/source/batch_002_incremental.csv \
  --workdir practice/output/m09_incremental --replay
```

The replay still creates run evidence and stages four records, but target logic should classify all four as already-current/same and apply zero business changes. Serving remains nine rows / 84,870.00.

Contrast that with the normal route without --replay: because the file hash is already in successful state, the pipeline may skip the already-processed file entirely. Both behaviors are legitimate but mean different things. Skip says "this exact source delivery already completed." Replay says "intentionally re-evaluate it but keep the business effect idempotent."

Now look at state.json. The processed hashes identify successful source byte sets. They do not prove that every logical source change after some timestamp was captured. That is why the dbt incremental example later discusses timestamp selection separately.

For interview practice, explain the difference among rerun, replay, backfill and full rebuild. Use the project rather than dictionary definitions.

Common mistakes / recovery

- Moving the watermark before target commit.
- Using timestamp alone when ties exist.
- Calling a successful rerun idempotent without checking target counts/values.
- Resetting state manually without recording why.

Recovery: inspect state semantics, select the smallest replay scope, run against a copy/fresh target when possible, reconcile, then update normal state deliberately.

Ready-made revision

Incremental correctness = deterministic selection boundary + idempotent target write + atomic state advance + late/change strategy + reconciliation. "Only process new rows" is too vague.

Evidence / mastery

Save the actual command/query/code, observed counts/values or error, one validation control, one short teach-back, and the changed retry. PASS -> continue to the next lesson.

PARTIAL/UNCERTAIN -> targeted repair + changed retry.

DE09-08

Failure behavior, retries, quarantine and atomic state

Watch first

[1. Failure behavior, retries, quarantine and atomic state - retained visual](#)

OPTIONAL visual support - no independently verified bounded range. Learner book + official reference + supplied project/lab are the required route.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: Complete the prior M09 lesson (or M08 for DE09-01). Keep the matching review closed until you have saved an independent attempt.

[Instructor demonstration: DataTalksClub DE Zoomcamp - dbt Commands](#) Optional visual support: [build/run](#) operational context. No independently verified bounded chapter range is assigned; retry/quarantine/atomic-state reasoning is taught and proved in the local failure lab.

[Official/current reference: dbt commands](#)

Use rule: Watch/read for the idea and interface, then return to this book and the supplied project. Close the source before the independent task. The course project and expected controls below are authoritative for this module's assessment.

Failure behavior is part of the pipeline design

A production pipeline is defined as much by what it does when something breaks as by its happy path. The module deliberately injects three different failure classes: bad data, transient database lock and a failure after staging.

Bad data: do not retry unchanged facts forever

batch_bad.csv has five source rows. Four violate required/type/range rules, so the quarantine ratio is 0.80. With the classroom quality threshold of 0.25, the run becomes FAILED_QUALITY_GATE. The previous successful state remains unchanged. Retrying the same unchanged file will not fix missing customer IDs or negative prices; this needs source repair or a deliberate rule change.

Transient database lock: bounded retry can help

The lock simulator fails the first database attempt with database is locked, then succeeds on attempt two. The manifest records db_attempts: 2. This is a reasonable retry candidate because the same deterministic transformation may succeed after a brief contention period.

Failure after staging: state must not lie

The injected failure on batch 002 occurs after staging files exist but before target commit/state advance. The run is marked FAILED_INJECTED_AFTER_STAGING; serving remains at the prior successful state and the new source hash is not added to processed state. On recovery, the

batch can be attempted again without pretending it was already complete.

This is why state is written after the serving transaction succeeds. Advancing the checkpoint first can create silent data loss: the rerun believes the batch is complete while the target never received it.

Quarantine is not a trash can

A quarantined record needs a reason, source pointer and ownership/remediation route. If 80% of a batch is quarantined and the pipeline still publishes "success", operators may never notice that most business data disappeared. Quality thresholds should reflect business risk and be visible in run metadata.

Independent proof

Run one quality-gate failure, one simulated lock and one injected post-staging failure. For each, record: retry or no retry? serving changed? state changed? raw evidence preserved? what human/system action is needed next? You should produce three different answers.

Changed retry

A network call returns 503 twice then succeeds; a schema contract fails immediately; a unique-key conflict appears inside the serving transaction. Classify each as transient, deterministic input failure or integrity failure, then choose retry/quarantine/abort behavior.

Atomicity prevents half-success from becoming invisible

The serving write uses a database transaction. If one row creates an integrity error, the transaction can roll back rather than leaving the first half committed. Only after the transaction succeeds does the pipeline update processed state. This ordering creates a clear recovery point.

Not every side effect can share one database transaction. A real pipeline might write files, call APIs and update a warehouse. Then you need idempotent writes, staging/rename patterns, commit markers or orchestration-level recovery. The principle remains: make partial progress visible and make retry behavior deliberate.

Retry budget and observability

A retry policy should record attempt number, reason and final outcome. Exponential backoff without a maximum can still stall a pipeline. Retrying an integrity violation wastes time. Retrying a lock forever hides capacity/coordination problems. Bound retries and escalate when the condition persists.

Stuck? Ask whether repeating the exact same operation later could succeed without changing input/config. If yes, it may be transient. If no, fix/quarantine the cause first. Then ask whether the operation is idempotent before retrying.

Follow with me: three failure drills with three different responses

Quality failure. Load batch 001, then batch_bad.csv in the same work directory. Expected bad run: five source, one staged, four quarantined, ratio 0.80, FAILED_QUALITY_GATE; serving/state remain at the prior successful state.

Transient lock. In a new directory run batch 001 with --simulate-lock-once. Expected db_attempts is 2 and final status is success. The retry is bounded because the simulated database-lock

condition can plausibly clear.

Failure after staging. Load batch 001, then run batch 002 with --fail-after-staging. The staging artifact exists, but serving should remain six / 22,930.00 and state should not include batch002's hash. Recovery can safely rerun from known evidence.

After each drill, answer the same four questions: What evidence exists? Did target state change? Did checkpoint/state change? Is retrying identical input useful? The answers should differ across the three cases.

This is more important than writing a generic try/except. Reliability is the deliberate relationship among error classification, idempotency, transaction boundaries, evidence and state.

Common mistakes / recovery

- Retrying 400/401/schema errors as if time will fix them.
- Catching every exception and returning success/empty output.
- Updating state in a finally block regardless of transaction outcome.

Recovery: classify the failure, confirm whether the operation is safe to repeat, inspect target/state before retry, then keep the failed run evidence beside the repaired run.

Ready-made revision

Retry transient operations, not deterministic bad data. Quarantine explainably. Commit target work atomically. Advance state only after success. Keep failed-run evidence.

Evidence / mastery

Save the actual command/query/code, observed counts/values or error, one validation control, one short teach-back, and the changed retry. PASS -> continue to the next lesson. PARTIAL/UNCERTAIN -> targeted repair + changed retry.

DE09-09

Lineage, run metadata and auditability

Watch first

[1. Lineage, run metadata and auditability - retained visual](#)

OPTIONAL visual support - no independently verified bounded range. Learner book + official reference + supplied project/lab are the required route.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: Complete the prior M09 lesson (or M08 for DE09-01). Keep the matching review closed until you have saved an independent attempt.

[Instructor demonstration: DataTalksClub DE Zoomcamp - dbt Documentation Video use: OPTIONAL visual](#) support. No independently verified bounded chapter range is assigned. Use it only for documentation/lineage visuals; the learner book + official reference + run-manifest lab are the required route.

[Official/current reference: dbt documentation](#)

Use rule: Watch/read for the idea and interface, then return to this book and the supplied project. Close the source before the independent task. The course project and expected controls below are authoritative for this module's assessment.

Lineage lets you answer "where did this number come from?"

Data lineage connects a result to its upstream source and transformation path. Operational auditability adds run identity, timestamps, code/config context, counts, failures and state transitions so another engineer can reconstruct what happened.

Every Python run in this module receives a run_id. The raw copy gets a SHA-256 hash and manifest. Staging/quarantine files use the same run ID. The serving row stores source_run_id. The run manifest records source rows, staged/quarantined/discarded counts, applied/ignored rows, database attempts, final serving count and serving net sales.

Trace one real order

After batch 002, serving row O6007 has net sales 52,250.00 and a source_run_id pointing to the successful incremental run. From that run manifest you can identify batch_002_incremental.csv and its source hash. From the raw copy you can see the original O6007 fields. The transformation rule explains 1 *

$55000 * (1 - 0.05).$

That is a defensible lineage story. "It came from Python" is not.

Audit evidence needs stable meaning

Do not fill logs with secrets or giant raw payloads. Record identifiers, counts, durations, contract versions, hashes and failure categories. A run ID should be unique. A source hash should identify bytes. A model/test name should identify logic. If the same label means different things across runs, the audit trail becomes misleading.

dbt lineage is related but not identical

source() and ref() create a transformation dependency graph. dbt documentation can show which models depend on which inputs. That graph explains logical transformation lineage. Your run manifests explain operational execution evidence. A mature system often needs both.

Independent proof

Pick O6007 and produce a short audit packet: serving row -> source_run_id -> run manifest -> source file/hash -> raw record -> calculation. Then pick the failed schema-drift run and show why no serving row should exist for it even though raw evidence exists.

Changed retry

Imagine a downstream analyst reports an incorrect total but only provides the date. Design the minimum run metadata fields that would let you identify candidate runs, code/model version, source files and row-count controls without storing sensitive raw payloads in logs.

Auditability is a chain, and missing links matter

A source hash tells you which bytes arrived, but without a code/model version you may not know which logic transformed them. A run ID identifies execution, but without source identifiers it cannot reconstruct inputs. A model DAG explains dependencies, but without actual run results it cannot prove what executed that night. Good audit design links these pieces.

In a team, include enough metadata to answer: who/what triggered the run, which source batch/version, which transformation version, what target was written, which tests/controls passed, what failed, and where evidence lives. Avoid logging credentials or unnecessary personal data.

Reproducibility versus exact binary replay

Reproducible business results do not always mean byte-for-byte identical outputs. File compression metadata, timestamps or row physical order may differ. Define which invariants must match: business keys/values, schema, counts, totals and controlled state. Hash raw immutable evidence where byte identity matters.

Teach it back: distinguish source lineage, transformation lineage and operational execution metadata using one O6007 example.

Follow with me: create a one-order audit trail

Choose O6007 after the successful incremental run. Use inspect_serving.py to find its source_run_id. Open the matching JSON in runs/. Confirm the source file is batch_002_incremental.csv and note its SHA-256.

Next open the raw copy beneath that run ID and locate O6007. Its source values are customer C307, East, Laptop, qty1, unit_price55000, discount0.05. Recalculate 52,250.00. You now have a full chain from business output to source bytes and rule.

Do the same exercise for the schema-drift file. You should find raw evidence and a failed manifest but no corresponding serving change. This proves that lineage can include failed attempts; an audit trail is not only a list of successful outputs.

When you later run dbt, save manifest.json/generated docs or a DAG screenshot/text as transformation-lineage evidence. Do not confuse that static graph with the run manifest from the Python pipeline. One explains dependencies; the other explains a specific execution.

Common mistakes / recovery

- Logging secrets/raw sensitive payloads "for audit."
- Generating a run ID but not connecting it to target rows or source hashes.
- Confusing a static DAG with proof that a specific production run succeeded.

Recovery: choose one output row and trace every identifier backward. If any hop cannot be followed, add the missing metadata link rather than adding more generic logs.

Ready-made revision

Lineage = dependency/origin path. Auditability = enough durable execution evidence to reconstruct what ran, on which source, under which rules, with which counts/results/failures.

Evidence / mastery

Save the actual command/query/code, observed counts/values or error, one validation control, one short teach-back, and the changed retry. PASS -> continue to the next lesson.

PARTIAL/UNCERTAIN -> targeted repair + changed retry.

DE09-10

Build and defend a reliable batch pipeline

Watch first

[1. Build and defend a reliable batch pipeline - retained visual](#)

OPTIONAL visual support - no independently verified bounded range. Learner book + official reference + supplied project/lab are the required route.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: Complete the prior M09 lesson (or M08 for DE09-01). Keep the matching review closed until you have saved an independent attempt.

[Instructor demonstration: DataTalksClub DE Zoomcamp - dbt Project Structure Optional visual support](#): batch-pipeline consolidation. No independently verified bounded chapter range is assigned; close the video before the independent integrated build.

[Official/current reference: dbt best practices](#)

Use rule: Watch/read for the idea and interface, then return to this book and the supplied project. Close the source before the independent task. The course project and expected controls below are authoritative for this module's assessment.

Now operate the whole batch pipeline as one system

The integrated pipeline is the point where isolated ideas become engineering behavior. A trustworthy run must preserve source evidence, enforce schema/types, normalize deterministically, resolve or quarantine duplicates, reconcile counts, write serving state transactionally, record run metadata and advance state only after success.

The operating loop

1. Receive a source batch and calculate its content identity.
2. Land an immutable raw copy + start manifest.
3. Validate header, required values, types, ranges and UTC timestamps.
4. Transform accepted rows deterministically.
5. Resolve identity/version; quarantine ambiguity.
6. Reconcile source = staged + quarantine + discarded.
7. Quality gate the batch; fail deliberately when too much is unresolved.
8. Write serving in one controlled transaction; retry only known transient lock conditions.
9. Reconcile target count/important measures.
10. Commit state only after success and write the final run manifest.

Defend the known controls

For the clean route, you should be able to state without reopening the solution:

- batch 001 -> 6 serving rows, 22,930.00;
- batch 002 -> 9 serving rows, 84,870.00;
- replay batch 002 -> 9 rows, 84,870.00, zero applied rows;
- bad-data batch -> 80% quarantine, quality gate failure, prior state preserved;
- duplicate fixture -> 1 staged + 2 ambiguous quarantined + 1 older discarded;
- schema drift -> contract failure, no state advance;
- simulated lock -> success on database attempt 2.

These numbers are not for memorization. They are a compact proof that you understand how each control changes state.

Closed-book module practical

Use M09_MODULE_CHECKPOINT.md. Start in a fresh runtime directory. Do not copy the reference command sequence blindly. Before every run, predict source count and intended target effect. After every run, save the manifest and independently inspect serving state. Explain one failure path aloud in 60-90 seconds.

Changed retry

Use batch_fresh_retest.csv only after the first checkpoint attempt is saved. Its three rows should form a new serving state of three rows worth 61,980.00 in a clean runtime. The data and amounts differ so recall alone cannot pass.

Defending a pipeline means explaining failures, not only showing green output

In an interview or production review, you should be able to answer: What is the grain? What can be replayed? How do you know rows were not lost? What happens on schema drift? Which failures retry? When does state advance? How do you trace a serving row back to raw? Which late updates can still be missed?

A screenshot of "SUCCESS" answers none of these. Your evidence packet should include one successful run, one changed incremental run, one replay, one quality failure, one contract failure and one recovery. The variety demonstrates control over behavior rather than memorization of a command.

Performance comes after correctness boundary

This module uses small files and SQLite so you can observe every state transition. Later tools scale the same concepts. Do not optimize by removing raw evidence, reconciliation or tests. First identify the expensive step, then choose partitioning, pushdown, incremental scope or distributed compute while preserving correctness controls.

Teach it back: give a five-minute whiteboard defense of the project without reading code. Include at least one known limitation and how you would strengthen it in production.

Follow with me: treat the checkpoint like an operator shift handoff

Open `practice/assessments/M09_MODULE_CHECKPOINT.md` but keep reference code/reviews closed. Use a fresh output directory. Before every command, write the expected row/state effect. After each command, inspect raw manifest, staging/quarantine, serving and state.

Your evidence set should include at minimum:

- one clean first load;
- one incremental update/new-key load;
- one replay with unchanged target totals;
- one duplicate/conflict case;
- one schema/quality failure;
- one transient retry or injected-failure recovery;
- one concise runbook explaining how to rerun safely.

Then take the fresh changed checkpoint with `batch_fresh_retest.csv` in a clean runtime. Predict the three net sales values before execution. Final target control should be three rows / 61,980.00. Because the IDs/prices/discounts differ from the main fixture, copying remembered answers cannot pass.

A strong portfolio README can later summarize this project with architecture, failure table, controls and one known limitation. Do not dump every internal course answer into the public portfolio; show engineering decisions and reproducible evidence.

Common mistakes / recovery

- Presenting only the happy path in a portfolio.
- Hiding quarantined/failed runs to make the project look clean.
- Claiming production-ready because the dataset is small and the script completed.

Recovery: defend one success, one incremental update, one replay, one contract failure and one transient recovery. Name the scale/runtime limitations explicitly.

Ready-made revision

A pipeline is not "read -> clean -> write". It is evidence -> contract -> deterministic transform -> identity -> reconciliation -> controlled commit -> state -> audit -> safe rerun.

Evidence / mastery

Save the actual command/query/code, observed counts/values or error, one validation control, one short teach-back, and the changed retry. PASS -> continue to the next lesson.

PARTIAL/UNCERTAIN -> targeted repair + changed retry.

DE09-11

dbt-style project anatomy, sources and dependency graph

Watch first

[1. dbt-style project anatomy, sources and dependency graph - retained visual](#)

OPTIONAL visual support - no independently verified bounded range. Learner book + official reference + supplied project/lab are the required route.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: Complete the prior M09 lesson (or M08 for DE09-01). Keep the matching review closed until you have saved an independent attempt.

[Instructor demonstration: DataTalksClub DE Zoomcamp - dbt Project Structure Video use: OPTIONAL visual](#) support. No independently verified bounded chapter range is assigned. Do not watch an entire recording merely to complete the lesson; inspect the supplied dbt project and official reference as the required route.

[Official/current reference: dbt ref function](#)

Use rule: Watch/read for the idea and interface, then return to this book and the supplied project. Close the source before the independent task. The course project and expected controls below are authoritative for this module's assessment.

dbt makes transformation dependencies explicit

The Python project proves reliability mechanics directly. dbt adds a structured way to express warehouse transformations as a graph of SQL models, sources, tests and metadata. In the supplied local project, DuckDB plays the analytical database so you can practise without a paid cloud account.

The project files have distinct jobs

- `dbt_project.yml` names the project and sets default materializations by folder.
- `profiles.yml` (copied locally from the example) tells dbt how to connect to `m09_dbt.duckdb`.
- `models/sources.yml` declares the raw source table.
- `models/staging/stg_orders.sql` normalizes raw fields.
- `models/intermediate/int_orders_deduped.sql` keeps the latest unambiguous order version in this
- controlled fixture.
- `models/marts/fct_orders.sql` calculates serving `net_sales`.

`setup_raw_duckdb.py` creates schema `raw` and table `raw.raw_orders` with ten order-version rows from batch 001 + batch 002. The source YAML declares schema `raw`, so { {

`source('raw','raw_orders') }}` points to the table actually created. This explicit source boundary is important; a mismatched schema name would make a perfectly valid model fail at runtime.

Ambiguous latest timestamps are a blocking contract, not an arbitrary window winner

The dbt project now makes the Python pipeline's rule executable. `int_order_conflicts` finds business keys with more than one row at the latest `updated_at`. `int_orders_deduped` includes only keys whose latest timestamp has exactly one candidate.

`tests/assert_no_ambiguous_latest_orders.sql` returns conflict rows, so a clean dbt build/dbt test must fail when the source does not provide a trustworthy tie-break. Do not repair that failure with file order or a timestamp-only `row_number()`.

`source()` and `ref()` are graph edges

`source('raw','raw_orders')` tells dbt that staging depends on an external declared source. `ref('stg_orders')` tells dbt that the intermediate model depends on staging. dbt resolves relation names and builds the DAG from these references. Hard-coding physical table names everywhere hides dependencies and makes environment changes harder.

The logical graph is: `raw.raw_orders -> stg_orders -> int_orders_deduped -> fct_orders`. The

incremental fact is another downstream consumer of `int_orders_deduped`.

Runtime truth in this review build

This project has passed static structure/SQL/YAML/Jinja checks, but live dbt/DuckDB execution remains a learner-PC proof. Therefore "project structure passes" is not the same as "dbt build passed". On your learner PC, create a venv, install `practice/requirements.txt`, run `python setup_raw_duckdb.py`, copy

```
profiles.yml.example to profiles.yml in the project folder, then use dbt debug --profiles-dir ., dbt
parse --profiles-dir . and dbt build --profiles-dir ..
```

Independent proof

Before running dbt, draw the expected graph from `filenames/Jinja`. After dbt parse/build, compare the actual graph/relations. Explain why `source()` is not the same as `ref()`. Save the command output and one screenshot/text artifact of the DAG or compiled model list.

Changed retry

Move `fct_orders.sql` to a different folder and change its default materialization deliberately. Predict what changes in physical relation type and what should not change in its upstream dependency graph.

Compiled SQL helps demystify dbt

`ref()` and `source()` are Jinja functions in authoring code; dbt compiles them into physical relation names for the active target. Inspecting compiled SQL is a useful recovery step when a model references the wrong schema or relation. Do not debug only the Jinja text when the actual database error mentions a compiled relation.

Folder names also affect configuration. In the supplied project, staging/intermediate default to views and marts default to tables via `dbt_project.yml`. A model can override that configuration. The folder is therefore not merely visual organization; it can carry execution defaults.

Source schema mismatch is a classic beginner failure

If setup creates `raw.raw_orders` but `sources.yml` declares schema `main`, the graph can parse while runtime lookup fails. The corrected package declares schema `raw`. When `dbt build` says a source relation does not exist, compare the database/schema/table that actually exists with the compiled source relation before rewriting model SQL.

Stuck? Run `dbt debug`, then `dbt parse`, inspect `target/manifest` or compiled SQL, confirm profile path/database, then confirm source schema. Change one thing at a time and save the first error.

Follow with me: walk the local `dbt` project in dependency order

From `practice/dbt_project/`, read `dbt_project.yml`, `profiles.yml.example` and `models/sources.yml`

before opening model SQL. Predict the physical database file and raw schema. Then inspect `setup_raw_duckdb.py`: it creates `raw.raw_orders` and loads ten versions from the first two batches.

After installing the requirements on the learner PC:

```
python setup_raw_duckdb.py
cp profiles.yml.example profiles.yml
dbt debug --profiles-dir .
dbt parse --profiles-dir .
dbt build --profiles-dir .
```

Expected setup message is `raw.raw_orders rows=10`. The exact `dbt` console formatting can change by version; your evidence is successful connection/parse/build plus the relations/tests produced.

Trace model dependencies in code: staging uses `source('raw','raw_orders')`; intermediate uses `ref('stg_orders')`; fact uses `ref('int_orders_deduped')`. If runtime says relation missing, check profile database path and source schema before changing transformation logic.

Keep `profiles.yml`, the local DuckDB file, `target/` and logs out of Git using the supplied `.gitignore`. The example contains no real secret, but practising separation now prevents later credential mistakes.

Common mistakes / recovery

- Hard-coding `database.schema.table` instead of using `source()/ref()` everywhere.
- Putting `profiles.yml` with real secrets into Git.
- Fixing relation-not-found errors by rewriting model logic before checking profile/database/schema.

Recovery order: `dbt debug` -> profile/path -> source declaration -> `dbt parse` -> compiled relation -> database object -> model SQL.

Ready-made revision

A `dbt` project turns transformation SQL into a dependency graph. `source()` anchors external/raw inputs; `ref()` connects `dbt`-managed models; configuration controls materialization without rewriting every physical relation name.

Evidence / mastery

Save the actual command/query/code, observed counts/values or error, one validation control, one short teach-back, and the changed retry. PASS -> continue to the next lesson.
PARTIAL/UNCERTAIN -> targeted repair + changed retry.

DE09-12

Executable data tests, quality severity and CI boundaries

Watch first

[1. Executable data tests, quality severity and CI boundaries - retained visual](#)

OPTIONAL visual support - no independently verified bounded range. Learner book + official reference + supplied project/lab are the required route.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: Complete the prior M09 lesson (or M08 for DE09-01). Keep the matching review closed until you have saved an independent attempt.

[Instructor demonstration: DataTalksClub DE Zoomcamp - dbt Tests Video](#) use: [OPTIONAL visual support](#). No independently verified bounded chapter range is assigned. The required route is the learner book + official data-test reference + deliberate local failing-test/repair exercise.

[Official/current reference: dbt data tests](#)

Use rule: Watch/read for the idea and interface, then return to this book and the supplied project. Close the source before the independent task. The course project and expected controls below are authoritative for this module's assessment.

Data tests are executable assertions, not decorative documentation

A description saying "order_id should be unique" is useful, but a test that fails when it is not unique is stronger. The supplied dbt project declares not-null/unique/accepted-value tests in YAML and includes a singular SQL test that returns rows only when net_sales < 0.

In dbt, a data test passes when its query returns zero failing rows. Generic tests express reusable assertions such as not_null, unique, relationships or accepted_values. Singular tests are ordinary SQL files that return violations of a project-specific business rule.

Severity belongs to risk

Not every quality observation should block publication. A duplicate primary business key is usually an error. A noncritical freshness warning may be allowed temporarily while alerting an owner. dbt supports warning/error behavior; your CI policy decides whether warnings are promoted to failures. The important habit is to connect severity to business impact, not to silence annoying tests.

A test can be correct and still insufficient

unique(order_id) on serving catches duplicate keys, but it cannot prove that the *right* version won. net_sales >= 0 prevents one class of monetary error, but it cannot prove the grand total reconciles to source. Combine structural tests with reconciliation and domain-specific assertions.

Deliberately make a test fail

On the learner PC, after a clean dbt build, change one fixture or model so a known rule fails. For example, introduce an invalid region or duplicate key in the local raw table. Predict which test should fail before running it. Save the failure output, repair the cause, rerun and save the pass. Do not weaken/delete the test just to make CI green.

CI boundary

A simple pull-request/CI gate can run dbt parse and a selected dbt build/dbt test. Error-level failure blocks merge/deploy. Warnings are recorded according to team policy. Expensive full-history checks may run on a different schedule. CI is not "run everything always"; it is a deliberate risk boundary.

Independent proof

List every supplied generic/singular test and the failure it is meant to catch. Then add one new test relevant to the project, make it fail intentionally, fix the underlying data/model and rerun. Explain why the test is placed at that model layer instead of somewhere else.

Changed retry

A new source region Central is legitimate next month. Your accepted-values test fails. Decide whether the correct action is to edit source data, update the contract/test, quarantine the value or block release. State the owner/evidence required before changing the accepted list.

Put tests close to the contract they protect

A staging accepted-values test protects normalization/contract assumptions. A serving unique-key test protects serving grain. A reconciliation test may need a separate audit model because it compares aggregates across layers. Test placement communicates ownership and shortens debugging.

Relationships tests deserve careful interpretation. A missing dimension key can mean bad fact data, late dimension arrival or a deliberate unknown-member strategy. The test failure is evidence; the remediation depends on the model contract. Do not automatically delete orphan facts.

CI should be selective but meaningful

For a small project, dbt build may be cheap enough to run on every change. At scale, teams often select changed models plus dependencies, run lightweight checks on pull requests and broader builds on schedules. Whatever the strategy, the selection itself is a risk decision. A fast CI pipeline that never exercises changed logic provides false confidence.

Teach it back: name one test that should block release in this project and one condition that could reasonably warn first. Explain the business impact behind the severity.

Follow with me: turn a quality statement into a failing query

Start with the YAML tests: order_id not-null/unique on deduped/fact models, region accepted values on staging, and source key/timestamp not-null. Then inspect tests/assert_nonnegative_net_sales.sql. It returns rows whose net sales are negative, so zero returned rows means pass.

On the learner PC, run a clean build, then deliberately introduce one controlled violation. For example, insert an invalid region into the local raw fixture or temporarily weaken normalization

so Central reaches staging while the accepted-values contract still allows only East/West/North/South. Predict the exact failing test before running:

```
dbt build --profiles-dir .
```

Save the failure output. Repair the cause/contract intentionally and rerun. The valuable evidence is the pair: predicted failure -> observed failure -> reasoned repair -> pass.

Do not "repair" by deleting the test. If Central is actually a newly approved region, update the contract and test with evidence/owner approval. If it is source corruption, preserve/quarantine it. The test tells you a promise was broken; engineering decides whether the promise or the data should change.

Common mistakes / recovery

- Treating every test as severity error without considering business impact, or the opposite: downgrading
- serious key tests to warnings.
- Testing only final marts, making root-cause location harder.
- Making a test pass by filtering out bad rows with no quarantine/owner.

Recovery: inspect failing rows first, identify the contract the test protects, fix cause or deliberately update contract, then rerun and preserve both failure and repair evidence.

Ready-made revision

Test = executable expectation. Generic tests protect common structural rules; singular tests protect project-specific rules. Severity should match business risk. A green test suite is evidence, not proof of every possible correctness property.

Evidence / mastery

Save the actual command/query/code, observed counts/values or error, one validation control, one short teach-back, and the changed retry. PASS -> continue to the next lesson. PARTIAL/UNCERTAIN -> targeted repair + changed retry.

DE09-13

Incremental models, materialization, documentation and handoff

Watch first

[1. Incremental models, materialization, documentation and handoff - retained visual](#)

OPTIONAL visual support - no independently verified bounded range. Learner book + official reference + supplied project/lab are the required route.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Before starting: Complete the prior M09 lesson (or M08 for DE09-01). Keep the matching review closed until you have saved an independent attempt.

[Instructor demonstration: DataTalksClub DE Zoomcamp - dbt Models Optional visual support: dbt models/materializations and command-line workflow.](#) No independently verified bounded chapter range is assigned for either recording; use the official dbt docs + supplied project as the required route. Companion commands video: <https://www.youtube.com/watch?v=t4OeWHW3SsA>

[Official/current reference: dbt materializations](#)

Use rule: Watch/read for the idea and interface, then return to this book and the supplied project. Close the source before the independent task. The course project and expected controls below are authoritative for this module's assessment.

Materialization chooses how a model exists physically

A dbt model can be a view, table, incremental relation or other adapter-supported materialization. The supplied project uses views for staging/intermediate and tables for marts by default.

`fct_orders_incremental.sql` overrides this with `materialized='incremental'` and `unique_key='order_id'`.

A view is cheap to rebuild because it stores logic rather than a full copied result, but downstream queries execute the logic. A table stores a result and needs rebuild/update behavior. Incremental materialization updates only a selected subset and therefore carries state-like correctness assumptions. "Incremental is faster" is incomplete unless you can prove it does not miss or duplicate changes.

The supplied incremental predicate is intentionally explainable

```
{% if is_incremental() %}
where updated_at >= (
  select coalesce(max(updated_at), '1900-01-01') from {{ this }}
)
{% endif %}
```

The `>=` re-reads rows tied at the current maximum timestamp. `unique_key='order_id'` makes re-reading those keys safe for the exercise. If you used `>` and a new order arrived later with

exactly the previous max timestamp, it could be skipped.

But this is not magic late-data protection. A correction with an arbitrarily old `updated_at` still falls outside the boundary. A real design may use CDC/change sequence, a lookback window, partition rebuild or source reconciliation.

Documentation is part of handoff

Model/column descriptions, tests and dependency metadata help another engineer understand grain, semantics and ownership. dbt docs generate builds documentation artifacts from the project. The DAG explains logical lineage; your README/runbook should also explain setup, runtime assumptions, incremental boundary and known limitations.

Full learner-PC handoff route

1. install the compatible stable 1.x requirements;
2. build the raw DuckDB fixture;
3. run dbt debug and dbt parse;
4. run dbt build and inspect generated relations;
5. run/inspect tests including a deliberate failure/repair;
6. build the incremental model, add/change source rows and prove expected update behavior;
7. generate docs and inspect lineage/descriptions;
8. save commands, results, one failure/recovery note and the known late-data limitation.

Independent proof

Explain why staging is a view, the normal fact is a table, and the second fact is incremental in this learning project. Then run the local project and save relation types/counts plus test results. Add one new order tied at the current max timestamp and prove it is not skipped. Do not claim arbitrary late data is solved.

Changed retry

Change the incremental strategy to use a one-day lookback (conceptually or in a safe local variant). State the extra compute cost, the duplication/upsert requirement and the class of late updates it still cannot capture.

Full refresh is part of the incremental contract

Incremental models need a documented way to rebuild from scratch when logic changes, historical corrections arrive or state becomes untrustworthy. In dbt, a full-refresh route can rebuild an incremental relation according to current logic. Treat that as an operational action with cost and reconciliation, not a casual button.

Changing a model from view to table or incremental can change storage, freshness and failure behavior while the SQL text remains similar. Handoff documentation should therefore name materialization, unique key, incremental predicate, full-refresh/backfill procedure and expected tests.

Late-data windows are trade-offs

A 14-day lookback reprocesses more data every run, increasing compute but covering corrections within that window. A CDC change sequence may be more precise but depends on source capability and retention. Partition rebuilds can be simple for date-partitioned facts. Choose based on source semantics, cost and recovery guarantees rather than copying one predicate everywhere.

Stuck? Before editing incremental SQL, write three examples: brand-new row, update at current max timestamp, correction older than the boundary. State whether each should be selected and how the target deduplicates/upserts it. If you cannot answer, the incremental contract is not finished.

Follow with me: prove an incremental boundary instead of trusting its name

After a normal dbt build, inspect `fct_orders_incremental.sql`. On an incremental run, it selects source rows whose `updated_at` is greater than or equal to the current target maximum. `unique_key='order_id'` lets the adapter update/reconcile keys rather than blindly append duplicates.

Create a changed local fixture with a new order ID at exactly the current maximum timestamp. Run the raw setup/update step and then the incremental model. The new key should be considered because of `>=`. If the predicate were `>`, that tied row could be skipped.

Next create a correction for an older timestamp outside the boundary. Predict that the simple incremental predicate will miss it. That failure is intentional teaching evidence: the model needs a lookback/CDC/full-refresh/reconciliation strategy if such late corrections are part of the real source contract.

Generate docs:

```
dbt docs generate --profiles-dir .
```

Inspect model descriptions, tests and lineage. Finish your handoff note with materializations, unique key, incremental predicate, rebuild/full-refresh route and known late-data limitation. A new engineer should be able to reproduce the project without guessing what state means.

Common mistakes / recovery

- Copying one incremental predicate to every source.
- Forgetting a full-refresh/backfill plan.
- Assuming `unique_key` can repair a row that was never selected.
- Documenting columns but not incremental limitations.

Recovery: test new row, boundary tie, late correction and rerun separately. If one case is missed, change source/change contract or selection window before calling the model reliable.

Ready-made revision

Materialization is a physical execution/storage choice. Incremental logic is a correctness contract. Documentation + lineage + tests + runbook make a model handoff reviewable. Always state the late/change boundary honestly.

Evidence / mastery

Save the actual command/query/code, observed counts/values or error, one validation control, one short teach-back, and the changed retry. PASS -> continue to the next lesson.
PARTIAL/UNCERTAIN -> targeted repair + changed retry.